

FORGE — FDE ACADEMY · RUNNING LLMs IN PRODUCTION · ENTERPRISE AI

From Deploy Day to Cost Discipline

Buffer · Eval · Telemetry · Cost

Pods that stabilise the Azure build · an eval harness with LLM-as-judge & Cohen's Kappa · Application Insights token + p99 telemetry · caching, batching & the PTU vs PAYG decision

consolidation & buffer

LLM-as-judge

drift detection

token & p99

PTU vs PAYG

SESSION MAP

What we cover today



Consolidation & buffer

Pause pods stabilise the Azure build before deploy day

1



Building an eval harness

LLM-as-judge · golden sets · regression tests · drift & Cohen's Kappa

2



Telemetry

Application Insights · custom events · token counting · latency p99

3



Cost discipline

Caching · smaller-model fallback · batching · PTU vs PAYG

4



Catch-up & concept walk-through

Open time to consolidate everything covered so far

5

OUTCOMES

By the end, you will be able to...



Warm the cluster

Use pause pods and overprovisioning so a launch doesn't cold-start on day one.



Trust the pipeline

Stand up a multi-evaluator harness and read agreement, disagreement and drift.



Measure agreement

Compute Cohen's Kappa and know which metric fits which eval.



See inside the app

Emit token counts, custom events and p99 latency to Application Insights.



Cut the bill

Apply caching, model tiering, batching and the right PTU / PAYG call.



Connect it up

Tie build, eval, telemetry and cost into one operating discipline.

01

CONSOLIDATION & BUFFER

Stabilise before deploy day



THE PROBLEM

Cold starts on launch day



Reactive scaling is too slow

- Traffic hits; the cluster is full; the autoscaler must add a node.
- A new AKS node takes ~5–10 min: VM boot, kubelet register, image pull.
- Those minutes are lag, errors and lost revenue on the day that matters most.



Proactive: a warm buffer

- Keep spare, pre-warmed capacity reserved before the rush arrives.
- Preempting a placeholder pod takes under a minute — not 5–10.
- Config issues surface during the buffer window, not at go-live.

Consolidation & Buffer, in one line

Pack workloads efficiently (consolidation) and hold pre-warmed placeholder capacity (buffer) so the system is responsive and reliable the moment production traffic arrives.

MECHANISM

How pause pods keep capacity warm

1

Reserve

Low-priority 'pause' pods (PriorityClass -1) request CPU/memory but do nothing — holding a node open.

2

Preempt

A real, higher-priority pod needs space → the scheduler evicts a pause pod instantly (<1 min).

3

Run

The real workload takes the freed capacity immediately — no waiting for a VM.

4

Replenish

Evicted pause pods go Pending → the cluster autoscaler adds a node to restore the buffer.

buffer.yaml

```
# 1. a very low priority class
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: overprovisioning }
value: -1          # below real workloads
---
# 2. placeholder 'pause' pods
kind: Deployment
metadata: { name: buffer }
spec:
  replicas: 3      # size of the buffer
  template:
    spec:
      priorityClassName: overprovisioning
      containers:
      - name: pause
        image: registry.k8s.io/pause:3.9
        resources:
          requests: { cpu: '1', memory: 2Gi }
```

GETTING IT RIGHT

Consolidation & stabilisation checklist



Compute consolidation

Bin-pack small workloads onto shared nodes; buffer pods fill the 'last node' slack as controlled headroom, not waste.



Right-size the buffer

AKS formula: for a 30% safety margin + 40% growth, overprovision ~50%. Tune replicas + CPU so exactly N standby nodes stay warm.



Pre-pull images

A DaemonSet pulls app images on every node; smaller / multi-stage images cut node-ready time so buffers turn productive fast.



Stabilise before go-live

Let pods cache images, open DB connections and reach healthy state during the buffer window — surface startup bugs early.

Warm capacity is insurance: a small, always-on cost that buys a responsive, error-free launch and steadier resource utilisation.

02

BUILDING AN EVAL HARNESS

Can you trust the pipeline's judgment?



THE THESIS

Valid output $\neq$ trustworthy output

The JDQI Eval Observatory

- The app already tells a user whether their job description is good.
- The harness answers a harder question: can you trust the pipeline's own judgment?
- It runs four evaluators in parallel and surfaces where they agree — and where they don't.
- It makes each evaluator's failure mode visible, side by side.



The key demo moment

On a weak JD: the code check passes 7/7 (valid JSON, scores in range) — yet the LLM judge and the reference scorer both fail it.

"Valid JSON is not the same as trustworthy output."

A structurally perfect report can still rest on scores the JD text never supports.

ARCHITECTURE

Four evaluators, run in parallel



Code checker

Deterministic. Schema, score ranges, weighted-total math, evidence present. No LLM · <50ms.



LLM-as-judge

A Sonnet call scrutinises each score against a rubric — is it supported by the JD text? ~3s.



Reference scorer

Numeric delta vs a hand-verified gold set + ROUGE-1 overlap of suggestions. No LLM · <20ms.



Human eval

A reviewer sets a verdict + notes in the UI — the anchor the others are measured against.

Agreement is the signal

An orchestrator (ThreadPoolExecutor, max_workers=3) runs the automated three, then records an agreement matrix and a disagreement count. When evaluators agree you have signal — agreement on 'warn' is as useful as agreement on 'pass'. Divergence tells you which failure mode is in play.

EVALUATOR 2

LLM-as-judge: rubric over vibes

What the judge checks

- Dimension support: does each score cite specific content from the JD, or is the evidence generic?
- Vagueness flags: are the suggestions concrete, or 'add more detail about skills'?
- One-paragraph reasoning for the verdict — auditable, not a bare number.
- Same model class as the pipeline (Sonnet) — a structured rubric lets it catch its own family's misses.

llm_judge.py

```
# strict rubric -> JSON verdict
VERDICT rule:
    pass : >=5 dims supported and <=1 vague
    warn : 3-4 dims   or 2-3 vague flags
    fail : <=2 dims   or >=4 vague flags

{
  "dimension_support": {
    "skill_specificity": false, # unsupported
    "compensation": false },
  "unsupported_scores":
    ["skill_specificity","compensation"],
  "vagueness_flags":
    ["Add more detail about compensation"],
  "verdict": "warn"
}
# fail -> warn on bad JSON (never crash)
```

EVALUATORS 1 & 3 + CI

Golden sets & regression tests



Golden / reference set

Hand-verified 'right answers': per-dimension scores (0–5) and ideal suggestions. The yardstick every run is measured against.



Reference scoring

Mean absolute score delta vs gold + ROUGE-1 recall of suggestions. pass ≤ 0.5 delta & ≥ 0.4 overlap; warn ≤ 1.0 & ≥ 0.2 ; else fail.



Regression tests

A pytest suite pins behaviour: 8 code-checker cases, 5 reference cases, 3 mocked-judge cases. Break the contract → CI goes red.



Frozen scenarios

Weak / average / strong JDs cached as JSON so the demo — and the tests — are instant and reproducible every run.

Golden set = 'what good looks like', frozen. Regression tests = 'it still behaves', enforced in CI. Together they stop silent quality drift between releases.

DRIFT DETECTION

Is the judge still calibrated?

Kappa as a drift alarm

- Cohen's Kappa scores agreement between the LLM judge and the human reviewer — chance-corrected.
- Log each (llm_verdict, human_verdict) pair; recompute kappa after every new human label.
- Track it as a rolling metric per cohort (topic, language) — a falling kappa flags drift early.
- Below the band? The judge — or the rubric — needs recalibration before you trust it at scale.

≥ 0.80

Excellent

0.60 – 0.79

Good

0.40 – 0.59

Moderate

< 0.40

Poor — recalibrate

kappa.py

```
def compute_kappa(pairs):  
    # (p_o - p_e) / (1 - p_e)  
    # observed vs chance agreement  
    return 0.0 if len(pairs) < 2 else kappa
```

CHOOSING A METRIC

Agreement metrics enterprises use

Metric	Use it when...	Note
Cohen's Kappa	Two raters (LLM vs human), categorical verdicts	Chance-corrected; handles class imbalance
Fleiss' Kappa	Three or more raters on the same items	Generalises Cohen's to many raters
Krippendorff's α	Many raters, missing labels, any scale	Most flexible; the general case
Quadratic Weighted κ	Ordinal scores (0–5) where near-misses matter	Penalises big disagreements more
F1 / precision-recall	Imbalanced pass/fail detection tasks	No chance correction — pair with kappa
Raw % agreement	A quick sanity read only	Inflated by chance; never report alone

Rule of thumb: pick the metric before you look at the number — raw agreement flatters, kappa keeps you honest.

REFERENCE CHECK

Cohen's Kappa in the wild



Clinical safety eval

An LLM-judge/jury screens model replies to users showing signs of psychosis, validated against clinician consensus.

Interesting fact: Avg κ = 0.66 (Gemini 0.75). Kappa chosen over F1 precisely because it handles class imbalance.



Health-economics review

LLMs scored 110 papers against the CHEERS checklist (2,860 item-level judgments) vs two human reviewers.

Interesting fact: LLM–human 72–95% agreement, AUC up to 0.96 — while the two humans agreed only κ = -0.07 to 0.43 with each other. No LLM hallucinations.



OSINT risk scoring

Nine models scored real-world incident posts on a scheming rubric, using Quadratic Weighted Kappa vs a two-reviewer consensus.

Interesting fact: Human–human κ = 0.70; one model (Claude Opus) hit κ = 0.77 — exceeding the humans' own agreement.

Yes — Cohen's Kappa is used in production-grade, regulated evals. The recurring lesson: measure the LLM against humans, and remember humans often agree less than you'd think.

03

TELEMETRY

See inside every model call



FOUNDATIONS

LLM telemetry $\neq$ ordinary APM

What traditional APM misses

- HTTP latency and error rate are table stakes — they don't show token spend or model choice.
- Cost tracks tokens, not requests: two calls to the same endpoint can cost 100× differently.
- Non-determinism: the same prompt gives different output, so you must capture inputs and params to reproduce.
- You need to see inside the call: model, tokens, tool calls, finish reason.



App Insights + OTel GenAI

- Application Insights is Azure Monitor's APM; it ingests OpenTelemetry.
- The OTel GenAI semantic conventions standardise `gen_ai.*` attributes across OpenAI, Anthropic, LangChain.
- A new Agents (Preview) view renders model calls, tool calls and token spend per agent.
- Auto-instrumentation means little app code — the telemetry layer wraps the calls.

INSTRUMENTATION

Token counts, custom events, p99



Token counting

gen_ai.usage.input_tokens / output_tokens on every span → cost in near real-time.



Custom events

Emit an 'llm.usage' event with agent, model, tokens, cost to customMetrics + traces.



Latency p99

Record a duration histogram; query percentile(duration, 99), not just the average.



Trace context

Attach user.id / session.id; keep prompt text out of attributes (PII) — use span events.

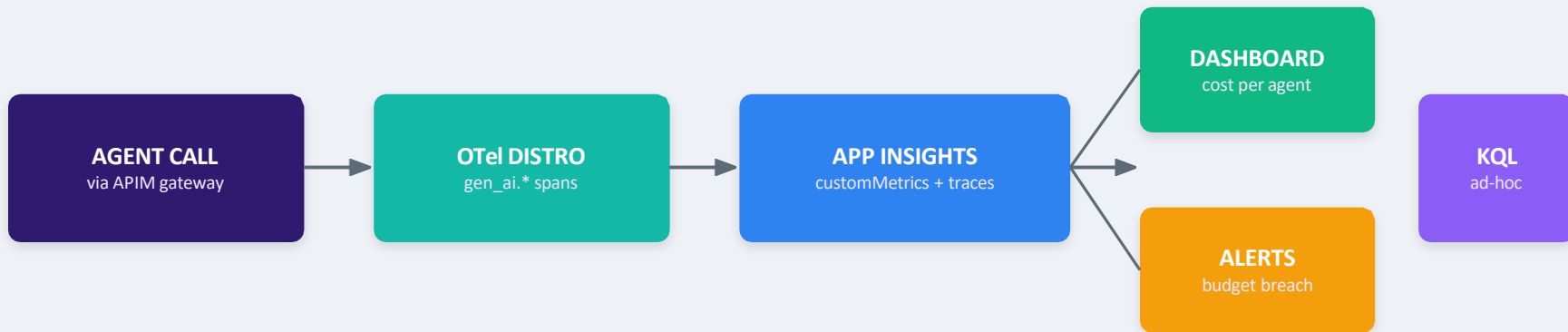
kusto.kql

```
// per-agent token + cost, last hour
traces
| where message == 'llm.usage'
| extend d = parse_json(customDimensions)
| summarize tok=sum(toint(d.total_tokens)),
    usd=sum(todouble(d.cost_usd))
by tostring(d.agent_name)

// p99 latency of chat calls
dependencies
| where name == 'chat'
| summarize
    p50=percentile(duration,50),
    p99=percentile(duration,99)
by bin(timestamp, 5m)
```

PATTERN

One pipeline: tokens → cost → alert



Why teams instrument this way

- Additive, not invasive — the telemetry wraps existing agents; no agent code changes.
- One flow answers three audiences: engineers (traces), platform (dashboards), finance (cost per outcome).
- Cost per user / API key / agent / tool call becomes a query, so model and prompt choices get grounded in real spend.

04

COST DISCIPLINE

Same output, a fraction of the bill



THE LEVERS

Four ways to cut LLM spend



Caching

Reuse repeated prompt prefixes — cached input costs 50–90% less and returns faster.



Smaller-model fallback

Route simple tasks to a nano/mini tier; escalate to a frontier model only when needed.



Batching

Send latency-tolerant work through the Batch API at 50% off — and stack it with caching.



PTU vs PAYG

Reserve capacity for steady baseline load; pay-as-you-go for burst and dev.

One more, free lever: shorten outputs

Output tokens cost 4–8× input. Cap max_tokens, ask for concise answers, and use structured (JSON-schema) output. Halving average output length halves your largest cost line — before you touch caching or PTUs.

LEVER 1

Prompt caching: stop re-paying for the prefix

How to earn the discount

- Azure caches repeated prefixes automatically after the first ~1,024 tokens (128-token steps).
- Put durable content first — system prompt, tools, few-shot, RAG context.
- Keep volatile content (the user's query) last, or a single early change busts the cache.
- Cached input is 50–90% cheaper: GPT-4.1 drops \$2.00 → \$0.50 per million (75% off).
- Cached tokens don't consume PTU capacity either — caching lowers the PTUs you need.

coding.py

```
# structure the prompt for cache hits
messages = [
    {'role': 'system',      # STABLE prefix
     'content': SYSTEM + TOOLS + FEWSHOT},
    {'role': 'user',       # VOLATILE, last
     'content': user_query},
]

resp = client.chat.completions.create(
    model='gpt-4.1',
    messages=messages,
    prompt_cache_key='jd-observatory-v1',
) # bias routing -> higher hit rate

# watch this like a health metric:
resp.usage.prompt_tokens_details.cached_tokens
```

LEVERS 2 & 3

Model tiering & batching



Smaller-model fallback

- Tier by difficulty: a nano/mini model for classification, tagging, routing.
- Escalate to a frontier model only when the small one is low-confidence or out of scope.
- Nano tiers are ~\$0.05–0.10 per million input — often 20–50× cheaper than a frontier model.
- A cheap router in front of an expensive model is the highest-leverage tiering pattern.



Batching

- The Batch API returns within 24h at 50% off standard pricing.
- Ideal for nightly jobs, document classification, embeddings, content pipelines — anything not real-time.
- Example: o3-mini drops from \$4.40 → \$2.20 per million tokens on Batch.
- Stacks with caching: batch 50% off + cache reads can reach ~95% off the repeated portion.

LEVER 4

PTU vs PAYG: the capacity decision

The decision rule

- PAYG (Standard): pay per token — flexible, best for variable or low volume.
- PTU (Provisioned): reserve capacity by the hour — predictable latency, lower effective rate on steady load.
- Break-even: PTU wins above ~50% sustained utilisation (~150–200M GPT-4o tokens/month).
- Validate against 30–60 days of real PAYG telemetry — measure P95 hourly throughput, don't trust the calculator.
- Hybrid: PTU for the baseline, PAYG for burst; spillover routes 429s to Standard.

Microsoft worked example (GPT-4o)

25 PTUs, reserved	\$6,500 / mo
-------------------	--------------

...delivers	62,500 in + 50,000 out TPM
-------------	----------------------------

Same load on PAYG	\$28,732 / mo
-------------------	---------------

Saving	≈ 4× cheaper
--------	--------------

Break-even	~7 days/mo of steady use
------------	--------------------------

Reserved (monthly / annual) adds up to ~70% off on-demand PTU.

EVIDENCE

The levers, in real numbers

50–90%

off cached input tokens

GPT-4.1 cached input: \$2.00 → \$0.50 / M (75% off), applied automatically.

60% → 87%

cache hit-rate jump

One production deployment, after adding `prompt_cache_key` to bias routing.

50% off

Batch API pricing

o3-mini \$4.40 → \$2.20 / M; stacks with caching toward ~95% on repeats.

≈ 4×

cheaper on PTU

25 PTUs \$6,500/mo vs \$28,732 PAYG for the same steady GPT-4o load.

Numbers are reported figures from Microsoft and provider docs (see references). Model against your own telemetry before committing spend.

05

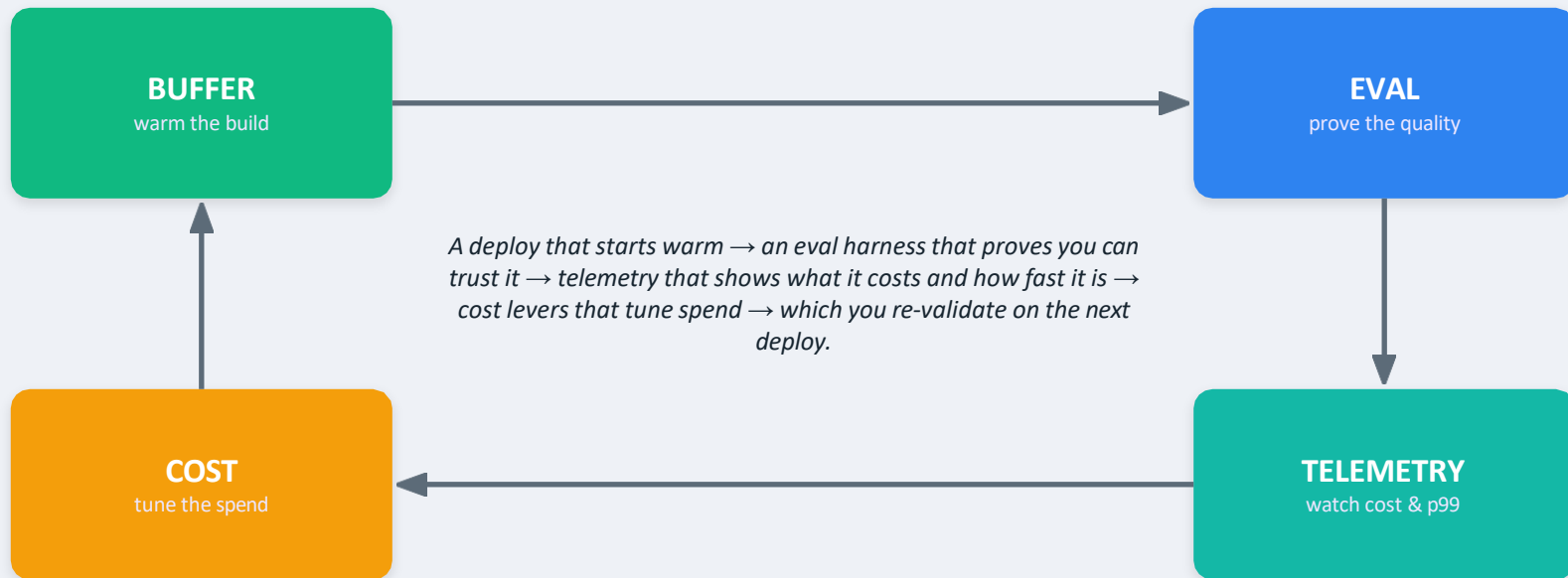
CATCH-UP & CONCEPT WALK-THROUGH

Tie the pieces together



THE BIG PICTURE

Four pillars, one operating loop



None of these stands alone: the cache hit-rate you tune in COST is a metric you read in TELEMETRY and a check you pin in EVAL.

OPEN CATCH-UP

Concepts worth a second pass



Preemption vs autoscaling

Why a pause pod frees space in <1 min but a new node takes 5–10.



Judge $\neq$ ground truth

The LLM judge is measured against humans; the golden set is the anchor.



Why kappa, not accuracy

Chance-correction is the whole point — raw agreement flatters.



customMetrics vs traces

Counters for dashboards/alerts; structured logs for ad-hoc KQL.



Prefix order = cache hits

Stable content first, volatile last — or the cache never fires.



PTU only on real data

Validate 30–60 days of P95 throughput before you reserve capacity.

Open floor: which of these do you want to see run live — the pause-pod preemption, the kappa panel, or a token/p99 KQL query?

06

REFERENCE

Glossary of terms & jargon



GLOSSARY · 1 OF 3

Azure infra & buffer

Consolidation & buffer Pack workloads tightly + hold warm spare capacity for launch.

Pause pod A do-nothing placeholder pod that reserves node capacity.

Overprovisioning Deliberately keeping spare capacity to absorb spikes.

PriorityClass A pod's scheduling priority; pause pods use a low/negative value.

Preemption Evicting a lower-priority pod to make room for a higher one.

Cluster autoscaler Adds/removes AKS nodes based on unschedulable pods.

Stabilisation Warming caches, images and connections before go-live.

Bin-packing Scheduling pods to use node capacity efficiently.

Cold start Latency while a new node provisions (~5–10 min on AKS).

Pre-pull (DaemonSet) Pulling images to every node ahead of time.

cluster-proportional-autoscaler Sizes the buffer to grow with the cluster.

AKS Azure Kubernetes Service — managed Kubernetes on Azure.

GLOSSARY · 2 OF 3

Eval & agreement metrics

Eval harness A system that scores a pipeline's output with several evaluators.

LLM-as-judge Using an LLM with a rubric to grade another model's output.

Golden / reference set Hand-verified 'right answers' used as the yardstick.

Reference scoring Numeric delta of pipeline scores vs the gold set.

ROUGE-1 Token-overlap recall between generated and reference text.

Regression test A pinned test that fails if behaviour changes unexpectedly.

Drift detection Spotting quality decay over time (e.g. falling kappa).

Agreement matrix Each evaluator's verdict, side by side.

Cohen's Kappa Chance-corrected agreement between two raters.

Fleiss' Kappa Kappa generalised to three or more raters.

Krippendorff's α Flexible agreement: many raters, missing data, any scale.

Quadratic Weighted κ Ordinal kappa that penalises larger disagreements more.

F1 score Precision-recall balance; no chance correction.

Chance correction Removing agreement expected by luck from a metric.

GLOSSARY · 3 OF 3

Telemetry & cost

Application Insights Azure Monitor's APM for app telemetry.

OpenTelemetry (OTel) Vendor-neutral standard for traces, metrics, logs.

GenAI semantic conventions Standard gen_ai.* attributes for LLM telemetry.

customMetrics Cumulative counters (tokens, cost) for dashboards/alerts.

traces Structured log records queried with KQL.

KQL Kusto Query Language for Azure Monitor Logs.

p99 latency The 99th-percentile response time — the slow tail.

Token counting Tracking input/output tokens per call for cost.

Prompt caching Reusing repeated prefixes for 50–90% cheaper input.

prompt_cache_key A hint that biases routing to raise cache hit-rate.

Batch API Async 24-hour processing at ~50% off standard rates.

Model tiering Routing easy tasks to cheaper/smaller models.

PTU Provisioned Throughput Unit — reserved model capacity.

PAYG / Spillover Pay-as-you-go; spillover routes PTU overflow to Standard.

REFERENCE

Sources & further reading



AKS performance & scaling — pause pods / overprovisioning

learn.microsoft.com



Leveraging low-priority pods for rapid scaling in AKS

techcommunity.microsoft.com



Monitor AI agents with Application Insights (Agents view)

learn.microsoft.com



Tracking every token — cost metrics for Foundry agents

techcommunity.microsoft.com



Prompt caching with Azure OpenAI

learn.microsoft.com



Do PTUs save you money? (worked example)

techcommunity.microsoft.com



Clinically-validated LLM-judge safety eval (Cohen's Kappa)

arxiv.org/abs/2604.02359



LLM vs human review of study quality (Cohen's Kappa)

jheor.org

RECAP

Run it warm, prove it, watch it, tune it

- Buffer: pause pods keep capacity warm so a launch preempts in seconds, not minutes.
- Eval: four evaluators + a golden set + Cohen's Kappa tell you whether to trust the pipeline.
- Telemetry: App Insights + OTEL GenAI make tokens, cost and p99 visible per agent.
- Cost: caching, model tiering, batching and the PTU/PAYG call cut the bill without cutting quality.
- They form one loop — each pillar feeds the next, and you re-validate on every deploy.